nomad-ai-commerce 코드 리뷰

리뷰 일자: 2026-10-11 / 대상 경로: D:₩WorkSpace₩platform₩nomad-ai-commerce

1. 프로젝트 개요

- 한 줄 설명: 아마존 구매대행 + 국내 위탁판매 셀러를 위한 AI 커머스 자동화 플랫폼. 상품 수집(아마존 URL, 엑셀), AI 가공, 숏폼 렌더링, 블로그 발행, 다계정 마켓 업로드를 React Flow 워크플로우로 연결하는 것이 목표(PRD 15단계).
- 목적/대상 사용자: 1인 구매대행 셀러, 위탁판매 셀러. 플랜(무료/기본/프로/엔터) 구독형 SaaS.
- 문서 상태: PRD.md는 15단계 마이크로 PRD, .cursorrules는 "Next.js 14" 기준 폴더 구조를 기술하지만 실제 코드는 Next 16.2.4 / (app) 라우트 그룹 구조로 문서와 코드가 어긋나 있음. CLAUDE.md는 @AGENTS.md 한 줄, README.md는 create-next-app 기본 템플릿 그대로.

영역	기술
프론트	Next.js 16.2.4 (App Router), React 19.2, Tailwind CSS v4, shadcn/ui(Radix) 7종, @xyflow/react 12, lucide-react
백엔드	Next.js Route Handlers(app/api/*) 12개, Node fetch 기반 외부 호출
DB	Supabase PostgreSQL (lib/supabase/schema.sql 3 테이블 + RLS), 서버는 service_role 키로 직접 접근
인프라	미정(Vercel 가정). .env* gitignore. 테스트/CI 없음
AI	OpenAI gpt-4o(Responses API, 상품명 최적화 - 미연결), gpt-image-1(썸네일 - 미연결). 워크플로우 AI 노드는 모킹

- 현재 완성도 추정: 약 25%.
  - 근거: 화면 7개와 API 12개가 존재하지만, 실제로 동작하는 외부 연동은 Supabase insert, 아마존 HTML 정규식 파싱, 환율 API(현재 키 없이 호출 실패)뿐임. 마켓 업로드/숏폼 렌더링/알림/AI 가공은 전부 setTimeout 시뮬레이션(app/api/render-shorts/route.ts:17, app/api/execute-flow/route.ts:365-374). 결제는 window.alert(app/admin/dashboard/settings/page.tsx:71). 로그인은 "임시 로그인"만 활성(components/auth/LoginModal.tsx:95-103)인데 아래 3장 #4처럼 그 경로마저 레이아웃에서 막혀 대시보드 진입이 불가능한 상태.
- 핵심 기능 목록
  - 랜딩 + 로그인 모달(소셜 로그인 비활성, 임시 로그인)
  - 대시보드: 환율 카드(USD/CAD), 아마존 URL 수집 탭, 위탁판매 엑셀 업로드 탭, 트렌드(하드코딩)
  - 상품 저장소: 선택 업로드 UI + 진행률 폴링(인메모리 잡)
  - 통합 설정: 마켓 계정 CRUD(Supabase market_accounts)
  - 슈퍼 어드민 설정: 플랜/슬롯/결제 모달(UI만)
  - 워크플로우 캔버스: 노드 5종 + /api/execute-flow 순차 실행 엔진
  - 블로그 발행: 지역 뉴스형 초안 생성 후 임의 테이블 insert

2. 구조 분석

경로	역할	비고
middleware.ts	/dashboard, /workflow, /admin 보호. Supabase 세션 또는 nomad_session 쿠키 확인, /admin은 nomad_role 쿠키/ADMIN_EMAILS로 판별	쿠키 신뢰 문제(3장 #1)
app/(app)/layout.tsx	서버에서 getServerUser()로 재검증 후 AppShell 렌더	모의 세션을 인정하지 않음(3장 #4)

경로	역할	비고
app/(app)/dashboard/*	대시보드/상품/설정/블로그/업로드 페이지(전부 "use client")	서버 컴포넌트 활용 없음
app/(app)/workflow/page.tsx	React Flow 캔버스, nodes 배열을 그대로 /api/execute-flow에 POST	엣지 무시, default 타입 노드 포함(3장 #5)
app/(app)/workflow/BlogPostInsertNode.tsx, CustomProcessingNode.tsx	라우트 폴더 안에 둔 노드 컴포넌트	CustomProcessingNode는 미사용(죽은 코드), 위치도 components/flow/nodes가 맞음
app/admin/dashboard/settings/page.tsx	플랜/슬롯/결제/음원 업로드 UI(369줄)	전부 로컬 state, 서버 연동 없음
app/api/amazon-fetch/route.ts	아마존 URL 정규식 파싱 + 엑셀 대량 insert	SSRF, raw_html 저장(3장 #2)
app/api/execute-flow/route.ts	노드 순차 실행 엔진, 내부 API를 fetch(origin + path)로 재호출	394줄, 인증 없음
app/api/market-accounts/route.ts	마켓 계정 CRUD, service_role 사용	인증 없음, 모든 계정의 api_key/secret 평문 반환(3장 #3)
app/api/blog/post/route.ts	요청의 tableName으로 임의 테이블 insert	테이블명 주입(3장 #6)
app/api/auth/mock-login, mock-logout	모의 로그인 쿠키 발급	운영 환경에서도 동작
app/api/billing/plan-check	쿠키 nomad_plan으로 플랜 판정	클라이언트 조작 가능
app/api/thumbnail/route.ts	gpt-image-1 썸네일 합성	호출하는 UI 없음, size 값 오류
lib/supabase/{client,server,admin}.ts	Supabase 클라이언트 3종	그런데 API 라우트 3곳이 이를 쓰지 않고 모듈 스코프에서 createClient 중복 생성
lib/utils/market-upload-jobs.ts, mock-market-accounts.ts	globalThis 인메모리 저장소	후자는 미사용, 전자는 서버리스에서 요청마다 사라짐
lib/product-utils.ts	금지어 필터 + GPT-4o 상품명 최적화	어디서도 import 안 됨(죽은 코드) + 정규식 버그
lib/supabase/schema.sql	users / market_accounts / billing_events + SELECT RLS	코드가 쓰는 amazon_products, dropship_inventory, banned_words, local_news_blog_posts 테이블 정의 없음

데이터 흐름 요약: 브라우저(클라이언트 컴포넌트)가 /api/(하위 경로)를 직접 fetch하고, API 라우트는 SUPABASE_SERVICE_ROLE_KEY로 Supabase에 직접 insert한다. 즉 RLS는 서버 경로에서 전혀 작동하지 않으며, API 라우트에 사용자 인증 검사가 하나도 없어 사실상 "누구나 service_role 권한으로 쓰기"가 가능한 구조다. 워크플로우 실행은 /api/execute-flow가 다시 자기 자신의 /api/(하위 경로)를 HTTP로 재호출하는 방식이라 인증 헤더 전파도 없고 서버리스 환경에서는 콜드스타트 비용이 노드 수만큼 중첩된다. 상태 저장이 필요한 업로드 잡은 globalThis에 두어 단일 프로세스 개발 서버에서만 동작한다.

3. 발견된 문제점

#	심각도	분류	위치	내용
1	높음	보안	middleware.ts:5-8, components/auth/AuthLauncher.tsx:32-33, app/api/auth/mock-login/route.ts:11	클라이언트가 임의 설정 가능한 nomad_role=SUPER_ADMIN, nomad_session=1 쿠키만으로 인증·관리자 권한 통과
2	높음	보안	app/api/amazon-fetch/route.ts:187-212, app/api/thumbnail/route.ts:40	사용자 입력 URL을 서버가 그대로 fetch(SSRF). 응답 HTML 전체를 raw_html 컬럼에 저장

#	심각도	분류	위치	내용
3	높음	보안	app/api/market-accounts/route.ts:16-46, lib/supabase/admin.ts:3-12	인증 없는 GET이 service_role로 전 사용자의 api_key, api_secret을 평문 반환. POST는 NOMAD_USER_ID 환경변수 1인 고정
4	높음	버그	app/(app)/layout.tsx:7-10 vs middleware.ts:49-52	미들웨어는 모의 세션을 통과시키지만 레이아웃은 Supabase 유저가 없으면 /?login=1로 리다이렉트. 소셜 로그인이 비활성(LoginModal.tsx:95)이라 현재 어떤 경로로도 대시보드 진입 불가
5	높음	버그	app/(app)/workflow/page.tsx:30-36,150 + app/api/execute-flow/route.ts:98-125	초기 노드 start/end의 type이 "default"인데 엔진 switch에 default 케이스가 미지원으로 처리되어 첫 노드에서 실행 실패. 엣지(연결 순서)도 무시하고 배열 순서로 실행
6	높음	보안	app/api/blog/post/route.ts:56-58,89,172-178	요청 바디 tableName으로 임의 public 테이블에 service_role insert 허용. 영숫자 검증만 있어 users, market_accounts 등 지정 가능
7	중간	보안	app/api/billing/plan-check/route.ts:12-18	플랜을 nomad_plan 쿠키에서 읽어 서버 판정에 사용. 클라이언트 조작으로 슬롯 제한 우회
8	중간	버그	lib/product-utils.ts:90,203-204	금지어 정규식에 ₩b 사용 - 한글은 ₩w가 아니므로 "화장품" 등 한글 금지어가 전혀 매칭되지 않음. escapeRegExp의 문자클래스 [.*+?^${}()/[₩₩]₩₩]도 잘못 이스케이프되어 특수문자 포함 금지어는 RegExp 생성 오류 가능
9	중간	버그	app/api/amazon-fetch/route.ts:231,241	템플릿 리터럴 내부 [₩s₩S]가 문자열 단계에서 [sS]로 바뀌어 요소 텍스트 추출 정규식이 실제로는 's'/'S' 한 글자만 매칭. #productTitle 파싱이 사실상 작동 안 함
10	중간	버그	app/api/exchange-rate/route.ts:3, app/(app)/dashboard/page.tsx:28	exchangerate.host는 현재 access_key 필수라 실패 응답을 그대로 success:true로 감쌈. base=KRW로 받아 역수 계산하는 우회 로직도 불필요
11	중간	버그	app/api/thumbnail/route.ts:61	gpt-image-1 size에 1080x1080 전달 - 지원값(1024x1024, 1536x1024, 1024x1536)이 아니어서 항상 400
12	중간	구조	lib/utils/market-upload-jobs.ts:15-22, app/api/market-upload/route.ts:10-23	업로드 잡을 globalThis에 저장하고 경과 시간으로 진행률을 "계산". 서버리스/다중 인스턴스에서 GET 폴링이 404. 실제 마켓 API 호출 없음
13	중간	구조	app/api/amazon-fetch/route.ts:48-50, app/api/blog/post/route.ts:30-32, lib/product-utils.ts:33-35	lib/supabase/admin.ts가 있는데도 3곳에서 모듈 스코프에 service_role 클라이언트를 중복 생성. SUPABASE_URL과 NEXT_PUBLIC_SUPABASE_URL 두 이름이 혼용
14	중간	구조	app/api/execute-flow/route.ts:376-394	내부 API를 HTTP 자기호출. 인증 쿠키 미전파, origin 스푸핑 가능, 실행 시간 중첩
15	중간	설정	lib/supabase/schema.sql	코드가 사용하는 amazon_products, dropship_inventory, banned_words, local_news_blog_posts 스키마 부재. users.role enum이 5개인데 코드 타입 UserRole은 2개
16	낮음	품질	lib/product-utils.ts, lib/utils/mock-market-accounts.ts, app/(app)/workflow/CustomProcessingNode.tsx, app/components/LogoutButton.tsx, app/(auth)/login/page.tsx	import되지 않는 죽은 코드. LogoutButton은 SidebarGnb/AuthLauncher와 로그아웃 로직 3중 중복
17	낮음	품질	app/(app)/dashboard/products/page.tsx:32-36, app/(app)/dashboard/page.tsx:20-24	상품 목록/트렌드가 하드코딩 더미. 수집한 amazon_products를 읽는 화면이 없음

#	심각도	분류	위치	내용
18	낮음	품질	app/admin/dashboard/settings/page.tsx:42-45, app/(app)/dashboard/settings/page.tsx:20-21	플랜/슬롯/배송 템플릿/음원 목록이 전부 로컬 useState. 새로고침 시 소멸. 어드민/일반 설정 페이지에 같은 토글이 중복 구현
19	낮음	설정	.cursorrules:3-25, README.md, app/layout.tsx:17	문서가 Next 14/다른 폴더 구조를 기술, 타이틀은 "Nomad AI Factory"로 프로젝트명 불일치. lib/utils/plan-policy.ts:16-45와 PRD 플랜 가격 출처 불명
20	낮음	성능	app/(app)/dashboard/settings/page.tsx:23-38	refreshAccounts와 useEffect 안에 같은 fetch가 중복 작성. 모든 페이지가 클라이언트 fetch로 워터폴 발생

심각도 높음 상세

#1 쿠키 기반 권한 위조. middleware.ts:5-8은 nomad_role 쿠키가 SUPER_ADMIN이면 관리자 검사를 통과시키고, :49-52는 nomad_session=1이면 로그인 검사를 통과시킨다. 이 쿠키는 AuthLauncher.tsx:32-33과 LoginModal.tsx:25-26에서 document.cookie로 브라우저가 직접 심으며, /api/auth/mock-login은 요청 바디의 role을 검증 없이 쿠키로 발급한다(mock-login/route.ts:11). 운영 환경 분기가 전혀 없어 배포 시 누구나 /admin/*에 들어갈 수 있다. 모의 로그인 경로 전체를 NODE_ENV !== 'production' 가드 아래로 옮기거나 제거하고, 역할은 Supabase users.role을 서버에서 조회해 판정해야 한다.

#2 SSRF 및 원문 HTML 저장. amazon-fetch/route.ts:187은 body.url을 호스트 검증 없이 fetch하므로 http://169.254.169.254/ 같은 내부 주소나 사설망 스캔에 악용될 수 있다. thumbnail/route.ts:40도 동일하다. 또한 응답 HTML 전체를 raw_html로 DB에 저장(:211)해 아마존 페이지 1건당 수백 KB가 쌓인다. 허용 호스트(amazon.*) 화이트리스트, 리다이렉트 제한, 응답 크기 제한, raw_html 저장 제거가 필요하다.

#3 마켓 API 키 전면 노출. market-accounts/route.ts GET/POST/PATCH 모두 supabase.auth.getUser() 호출이 없다. service_role 클라이언트로 market_accounts 전체를 읽어 api_key, api_secret을 그대로 응답하므로(:21,33-34) URL만 알면 모든 사용자의 마켓 자격증명을 받을 수 있다. 사용자별 격리도 NOMAD_USER_ID 환경변수 하나(:57)에 의존해 다중 사용자 구조가 아니다. 인증 유저의 users.id로 필터하고 비밀값은 마스킹 응답 + 서버측 암호화 저장이 필요하다.

#4 로그인 불가 상태. 미들웨어는 모의 세션을 허용하지만 (app)/layout.tsx:7-10은 getServerUser()가 null이면 /?login=1로 보낸다. Supabase 환경변수가 없으면 createClient()가 null을 반환해 항상 null이고, 있어도 모의 로그인은 Supabase 세션을 만들지 않으므로 결과는 같다. 유일하게 활성화된 "임시 로그인" 버튼이 대시보드로 보내자마자 다시 랜딩으로 튕기는 루프가 발생한다. 소셜 로그인 버튼이 disabled이므로(LoginModal.tsx:95,101) 현재 빌드는 사용자가 어떤 화면에도 진입할 수 없다.

#5 워크플로우 엔진 즉시 실패. 캔버스의 start/end 노드는 type: "default"(workflow/page.tsx:33,92)인데 엔진은 "start"만 처리하고 그 외는 success:false(execute-flow/route.ts:120-124)를 반환하며 첫 실패에서 전체를 500으로 중단한다(:73-82). 따라서 기본 템플릿을 그대로 실행하면 항상 "지원되지 않는 노드 타입: default"로 끝난다. 추가로 엔진은 엣지를 받지 않고 nodes 배열 순서로만 실행해 사용자가 캔버스에서 바꾼 연결이 반영되지 않는다.

#6 테이블명 주입. blog/post/route.ts:58은 payload.tableName을 ^[a-zA-Z0-9_]+$만 검사해 supabase.from(tableName).insert(...)에 넘긴다(:89). service_role이므로 RLS가 적용되지 않아 users, market_accounts, billing_events 등 어떤 public 테이블에도 임의 레코드를 쓸 수 있다. 허용 테이블을 상수로 고정하고 요청 바디에서 테이블명을 제거해야 한다.

4. 효율성 개선 제안

구조/아키텍처

- API 라우트 공통 가드 도입: lib/supabase/server.ts의 getServerUser()를 사용하는 requireUser() 헬퍼를 만들어 모든 app/api/*에서 첫 줄에 호출. service_role은 webhook/크론 등 서버 전용 경로에만 사용하고 사용자

요청은 anon 키 + RLS로 처리.

- 워크플로우 엔진을 HTTP 자기호출 대신 lib/engine/*.ts 함수 직접 호출로 전환. { nodes, edges }를 받아 위상 정렬 후 실행하고, 노드별 실행기(runAmazonCollect 등)를 라우트와 공유.
- 모의 인증 제거 또는 process.env.NODE_ENV !== 'production' && process.env.ENABLE_MOCK_AUTH === '1'로 격리. 역할/플랜은 쿠키가 아닌 DB(users.role, users.plan)에서 읽기.
- 노드 컴포넌트를 components/flow/nodes/로 통합(app/(app)/workflow/*Node.tsx 이동, CustomProcessingNode 삭제).

성능

- 대시보드/상품/설정 페이지를 서버 컴포넌트로 전환해 초기 데이터(마켓 계정, 수집 상품)를 서버에서 1회 조회. 클라이언트 폴링(products/page.tsx:63-72)은 Supabase Realtime 또는 잡 테이블 기반으로 교체.
- amazon-fetch의 raw_html 저장 제거, 응답 바디 크기 제한(예: 2MB), 정규식 파싱을 cheerio/linkedom 같은 파서로 교체.
- exchange-rate에 unstable_cache 또는 Route Segment revalidate 적용과 함께 신뢰 가능한 공급자(한국수출입은행 API)로 교체.

개발 생산성 (빌드, 타입, 테스트, 중복 제거)

- Supabase 클라이언트 생성 단일화: lib/supabase/admin.ts만 사용, SUPABASE_URL/NEXT_PUBLIC_SUPABASE_URL 이름 통일.
- schema.sql에 누락 테이블(amazon_products, dropship_inventory, banned_words, blog_posts)과 market_upload_jobs 추가, supabase gen types로 타입 생성해 row.id as string 캐스팅 제거.
- 죽은 코드 삭제(lib/product-utils.ts는 금지어 로직을 수정 후 execute-flow의 customProcessing에 연결하는 편이 PRD 8단계에 부합).
- 최소 테스트: replaceBannedWords, computeJobProgress, parseMetaTag에 대한 Vitest 단위 테스트. API 라우트 인증 가드 테스트.
- 문서 정리: .cursorrules/README.md를 실제 구조로 갱신, app/layout.tsx 타이틀을 프로젝트명으로.

운영/배포

- .env.example 추가(필수 변수 10개 정리). 빌드 시 필수 env 검증(lib/env.ts + zod).
- 운영 배포 전 mock-login/mock-logout 라우트 제거 확인을 CI 체크로.
- 외부 호출(OpenAI, 아마존 fetch)에 타임아웃(AbortSignal.timeout)과 에러 로깅 추가.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	모의 인증 제거/격리 및 middleware.ts 재작성: 쿠키 신뢰 삭제, Supabase 세션만 인정, 관리자 판정을 users.role DB 조회로 전환, (app)/layout.tsx와 판정 로직 일치시키기	P0	M	Fable 5.1	인증·권한 로직, 실수 시 전면 권한 상승
2	전 API 라우트 인증 가드(requireUser) 도입 + market-accounts를 사용자별 필터/비밀값 마스킹/암호화 저장으로 변경, NOMAD_USER_ID 제거	P0	M	Fable 5.1	보안·데이터 격리 재설계, 다중 파일
3	blog/post 테이블명 주입 제거(허용 테이블 상수화), amazon-fetch/thumbnail SSRF 방어(호스트 화이트리스트, 크기/타임아웃 제한, raw_html 저장 제거)	P0	S	Fable 5.1	입력 검증 보안 수정, 실수 비용 큼
4	Supabase 소셜 로그인(Google/Kakao) 실제 연결: LoginModal 버튼 활성화, signInWithOAuth + /auth/callback 흐름 검증, 로그인 후 users 프로필 upsert	P0	M	Opus 5.5	정의가 명확한 API 연동 작업

#	작업	우선순위	예상 규모	권장 모델	선정 이유
5	워크플로우 엔진 수정: default 노드 처리, {nodes, edges} 수신 후 위상 정렬 실행, HTTP 자기호출을 함수 직접 호출로 교체	P1	M	Opus 5.5	2~4개 파일 리팩토링, 로직 명확
6	schema.sql 보강(누락 테이블 4종 + 업로드 잡 테이블 + INSERT/UPDATE RLS), supabase gen types 도입, API 라우트 캐스팅 제거	P1	M	Fable 5.1	데이터 모델 재설계
7	마켓 업로드 잡을 globalThis에서 DB 테이블 기반으로 전환, 진행률 폴링을 Realtime 구독으로 교체	P1	M	Opus 5.5	상태관리 정리, 중간 규모
8	lib/product-utils.ts 정규식 버그 수정(한글 경계 처리, escapeRegExp), 단위 테스트 작성, customProcessing 노드에 실제 연결	P1	S	Opus 5.5	중간 난이도 버그 + 테스트 설계
9	환율 API 교체(한국수출입은행 또는 키 기반 공급자) 및 응답 실패 시 success:false 반영, 썸네일 size 값 수정(1024x1024)	P1	S	Sonnet 5.5	단순 버그·설정 수정
10	죽은 코드 삭제(mock-market-accounts.ts, CustomProcessingNode.tsx, LogoutButton.tsx, (auth)/login), 노드 컴포넌트 components/flow/nodes로 이동, 로그아웃 로직 단일화	P2	S	Sonnet 5.5	반복적 정리 작업
11	대시보드/상품/설정 페이지 서버 컴포넌트 전환 + 수집 상품(amazon_products, dropship_inventory) 실제 목록 표시, 더미 데이터 제거	P2	M	Opus 5.5	데이터 페칭 구조 정리
12	어드민 설정(플랜/슬롯/배송 템플릿/음원) 상태를 DB(users.plan, user_settings)와 Supabase Storage에 연결, 일반 설정 페이지와 중복 토글 통합	P2	M	Opus 5.5	명확한 CRUD 기능 구현
13	.env.example, lib/env.ts(zod 검증) 추가, .cursorrules/README.md/app/layout.tsx 메타데이터 갱신	P2	S	Sonnet 5.5	설정·문서 정리
14	결제(Stripe 또는 토스) Checkout 세션 + webhook으로 users.plan/billing_events 갱신 구현	P2	L	Fable 5.1	결제 로직, 실수 시 금전 피해

6. 종합 평가

- 강점
  1. PRD 15단계와 라우트/노드 구조가 1:1로 대응되어 어떤 화면이 어떤 단계인지 추적이 쉽다.
  2. any 사용이 0건이고 unknown 좁히기 패턴이 일관되어 타입 품질 자체는 양호하다.
  3. execute-flow의 노드별 실행기 분리, plan-policy.ts의 플랜 정책 상수화 등 확장 포인트가 미리 마련되어 있다.
- 약점
  1. 인증/권한이 클라이언트 쿠키에 의존하고 API 라우트에 인증이 전혀 없어, 현재 코드는 외부 노출이 불가능한 수준이다.
  2. 실제 동작하는 기능이 거의 없고(마켓 업로드·렌더링·AI·결제 모두 시뮬레이션), 유일한 진입 경로인 임시 로그인도 레이아웃 가드와 충돌해 막혀 있다.
  3. 스키마와 코드가 불일치하고, 죽은 코드·중복 클라이언트·인메모리 상태가 섞여 있어 서버리스 배포 시 바로 깨진다.
- 모델별 비율(작업 14개 기준): Fable 5.1 36% (5건) / Opus 5.5 43% (6건) / Sonnet 5.5 21% (3건)
- 권장 진행 순서
  1. 1단계(P0): 작업 1-3으로 인증·입력검증 구멍을 막고, 작업 4로 실제 로그인 경로를 연다. 이 단계 전에는 배포 금지.
  2. 2단계(P1): 작업 5-9로 워크플로우 엔진, 스키마, 업로드 잡, 금지어 필터, 환율을 "실제 동작"으로 전환.
  3. 3단계(P2): 작업 10-14로 정리·서버 컴포넌트 전환·설정 영속화·결제 연동.

- 주 담당 모델: **Fable 5.1**. 이 프로젝트의 병목은 UI가 아니라 인증·권한·데이터 격리·스키마 재설계이며, 쿠키 위조·SSRF·테이블명 주입처럼 실수 비용이 큰 보안 작업이 P0의 전부를 차지한다. 보안 골격이 잡힌 뒤의 기능 연결(소셜 로그인, 엔진, 잡 테이블)은 Opus 5.5로 넘기고, 정리·문구·설정은 Sonnet 5.5로 분담하는 것이 비용 대비 안전하다.